Ivett Anahi Zaragocin Mora

Tipos, tiempo y pulsadores

PROGRAMAR PARA EL HARDWARE · SESIÓN 2

Activación: ¿qué imprime?

```
uint8_t a = 0xCC;       // 1100 1100
Serial.println(~a, BIN);   // ¿?
```

`a` se promueve a int antes de aplicar `~` (ESP32: 32 bits): se invierten 32 bits, no 8.

a = 0xCC (1 byte)

1	1	0	0	1	1	0	0

~a (promovido a int, 32 bits)

1	1	1	1	1	1	1	1

× 3 bytes (24 unos)

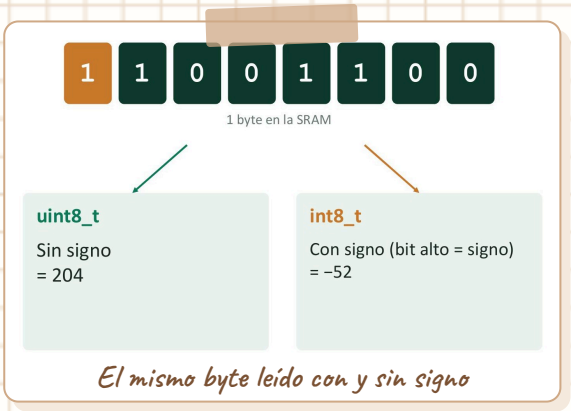
0	0	1	1	0	0	1	1

Salida: `11111111111111111111111100110011`. Uno (`int` de 16 bits): `1111111100110011`. Con `(uint8_t)~a`: `110011`.

Parte 1 · Tipos de datos

✦ Una variable es una caja en la SRAM

- *uint8_t*: sin signo, **204**.
- *int8_t*: con signo, **−52** (204 − 256).



El mismo byte leído con y sin signo

int no mide lo mismo en todas partes

Tipo (bytes)	AVR (Uno)	ESP32	Ojo
`char`	1 · con signo	1 · sin signo	usar `int8_t` / `uint8_t`
`short`	2	2	
`int`	2	4	¡cambia de tamaño!
`long`	4	4	
`float`	4	4 (FPU)	rápido en el ESP32
`double`	4 (= float)	8 (software)	lento en el ESP32
puntero	2	4	

stdint.h: el tamaño va en el nombre

Con signo	Rango	Sin signo	Rango
`int8_t`	-128 … 127	`uint8_t`	0 … 255
`int16_t`	-32 768 … 32 767	`uint16_t`	0 … 65 535
`int32_t`	≈ ±2,1 × 10^9	`uint32_t`	0 … ≈ 4,29 × 10^9

`uint16_t` = entero unsigned de 16 bits, tipo (_t). Usarlos en registros, máscaras, tramas y tiempos.

Desbordamiento



`uint8_t c = 253;` y `c++` *cinco veces: 253, 254, 255, 0, 1*

Sin signo

Da la vuelta. Comportamiento definido.

Con signo

Comportamiento indefinido. No confiar en él.

`int t = millis();`

Uno: falla a los ~32 s.
ESP32: falla a los ~24,8 días.
Usar `uint32_t`.

Demo: detector de plataforma

```
#define MOSTRAR(T) do { Serial.print(#T); Serial.print(": "); \
    Serial.print(sizeof(T)); Serial.println(" bytes"); } while (0)

void setup() {
  Serial.begin(115200);
  MOSTRAR(char); MOSTRAR(int); MOSTRAR(long);
  MOSTRAR(float); MOSTRAR(double); MOSTRAR(void*);

  uint8_t c = 253;                  // desbordamiento
  for (int i = 0; i < 5; i++) { Serial.print(c++); Serial.print(' '); }
  Serial.println();

  uint8_t a = 0xCC;                 // promoción
  Serial.println(~a, BIN);
  Serial.println((uint8_t)~a, BIN);
}
void loop() {}
```

En Wokwi (Uno y ESP32): `sizeof` difiere; el bucle da `253 254 255 0 1` .

Máscaras: el 1 también tiene tipo

✘ Riesgoso

```
uint32_t m = 1 << 31;  // int con signo
```

✔ Correcto

```
uint32_t m = 1UL << 31; // sin signo
```

Literales: `0x` hexadecimal · `0b` binario · `U` sin signo · `UL` unsigned long

Constantes con tipo

Forma	Qué es	¿Tipo?	Uso
`#define LED 16`	reemplazo de texto	no	solo por compatibilidad
`const uint8_t LED = 16;`	constante verificada	sí	
`constexpr uint8_t LED = 16;`	calculada al compilar	sí	la usaremos
`enum Modo { LENTO, RAPIDO };`	estados con nombre	sí	modos del juego

⚠ ESP32: una `const` global queda en la **flash**. AVR: se copia a la **SRAM**, salvo con `PROGMEM` o `F()`.

volatile y static

VOLATILE

El valor puede cambiar sin que el programa lo vea (**interrupción** o **hardware**). **No es atómico.**

STATIC

En función: conserva su valor entre llamadas. **En archivo:** privada.

⚠ Las locales viven en la **pila** de `loopTask` (8 KB): un arreglo local grande la desborda y el ESP32 se **reinicia**.

Parte 2 · El tiempo sin bloquear

✦ delay() bloquea



`loop()` *con* `delay(500)` *: una pulsación entre lecturas nadie la lee*

✦ millis(): mirar el reloj

```
uint32_t ahora = millis();

if (ahora - tAnterior ⩾ PERIODO) {
  tAnterior += PERIODO;

  // hacer algo
}
// el loop sigue: hay tiempo para leer botones
```

El patrón

1. Leer el reloj.
2. Restar.
3. Si toca: actuar y anotar.
4. Seguir con lo demás.

✦ Cuando millis() da la vuelta

248	250	252	254	0	2	4	6
	tAnterior			reloj de 8 bits: 255 → 0		ahora	

Con un contador de 8 bits la cuenta pasa de 255 a 0, pero 4 − 250 sigue dando 10 ms

✔ Restar

`ahora - tAnterior`

= 4 − 250 = **10** (módulo 256).

Es justo el tiempo que pasó.

✘ Sumar y comparar

`millis() > tAnterior + PERIODO`

La suma da la vuelta y la condición se cumple **antes de tiempo**.

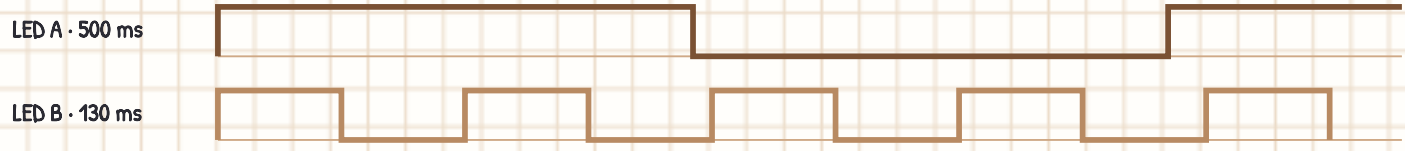
Ejemplo: dos LEDs a ritmos distintos

```
constexpr uint8_t LED_A = 16, LED_B = 17;
constexpr uint32_t PERIODO_A = 500, PERIODO_B = 130;    // ms
uint32_t tA = 0, tB = 0;
bool estadoA = false, estadoB = false;

void setup() { pinMode(LED_A, OUTPUT); pinMode(LED_B, OUTPUT); }

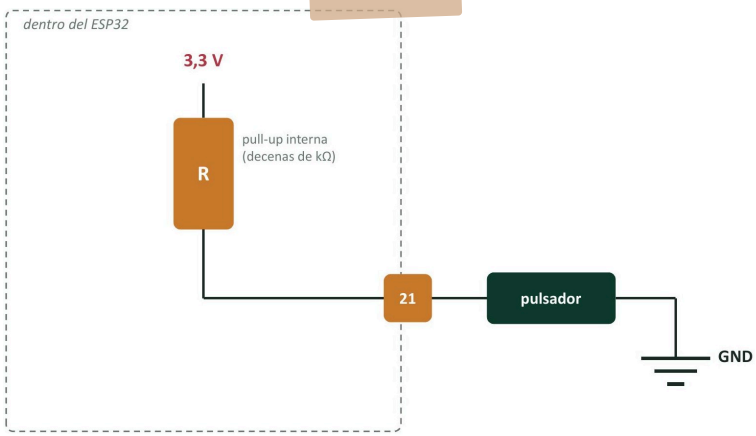
void loop() {
  uint32_t ahora = millis();
  if (ahora - tA ⩾ PERIODO_A) {
    tA += PERIODO_A; estadoA = !estadoA; digitalWrite(LED_A, estadoA);
  }
  if (ahora - tB ⩾ PERIODO_B) {
    tB += PERIODO_B; estadoB = !estadoB; digitalWrite(LED_B, estadoB);
  }
}
```

LEDs en GPIO16 y GPIO17, 330 Ω a GND. El estado vive en variables, no en el pin.



Parte 3 · Leer un pulsador bien

✦ Pull-up: nivel definido en el pin



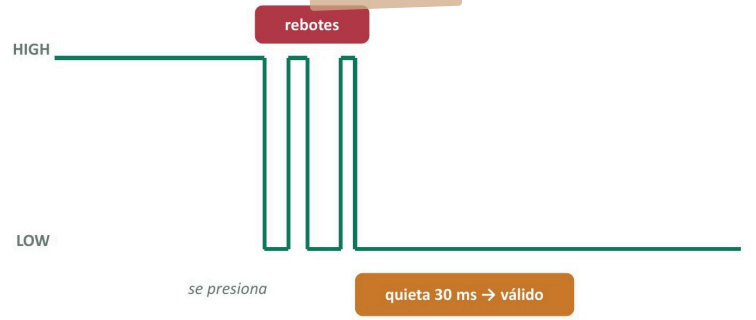
Pull-up interna del ESP32, pulsador en el GPIO 21

Pulsador	Lectura
suelto	HIGH (1)
presionado	LOW (0)

GPIO 34–39: sin pull-up interna.

Se activa con `INPUT_PULLUP`.

✦ Rebote y antirrebote

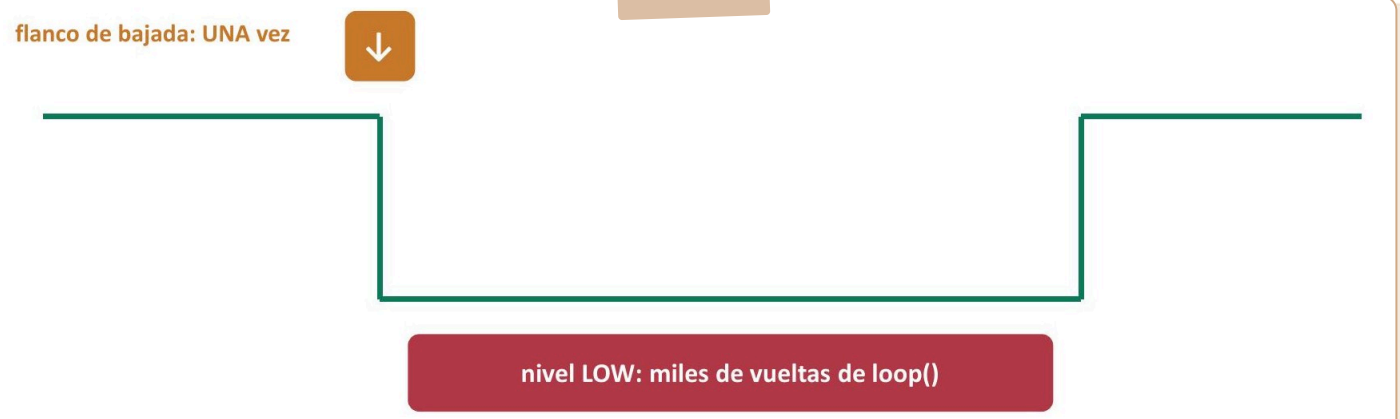


Al presionar, la señal oscila antes de quedar en LOW

Antirrebote aceptar el cambio solo si la señal queda **quieta 20–50 ms** (aquí, 30 ms).

⚠ No usar `delay(200)`: bloquea el programa.

✦ Nivel frente a flanco



Un toque dura ~200 ms: miles de vueltas con el nivel en LOW, pero el flanco de bajada ocurre UNA vez

✘ Por nivel

```
if (digitalRead(BTN) == LOW) contador++;
```

✔ Por flanco

Actuar solo al pasar de **HIGH a LOW**.

Código: antirrebote y flanco

```
constexpr uint8_t PIN_BTN = 21;
constexpr uint32_t T_REBOTE = 30;                  // ms
bool lecturaAnterior = HIGH, estadoEstable = HIGH;
uint32_t tUltimoCambio = 0; uint16_t pulsaciones = 0;

void setup() { Serial.begin(115200); pinMode(PIN_BTN, INPUT_PULLUP); }

void loop() {
  bool lectura = digitalRead(PIN_BTN);                    // 1) leer
  if (lectura ≠ lecturaAnterior) {                    // 2) ¿se movió? reiniciar
    tUltimoCambio = millis(); lecturaAnterior = lectura;
  }
  if (millis() - tUltimoCambio ⩾ T_REBOTE &&       // 3) ¿quieta 30 ms?
      lectura ≠ estadoEstable) {
    estadoEstable = lectura;                              // cambio real
    if (estadoEstable == LOW) {                        // 4) ¿flanco de bajada?
      pulsaciones++; Serial.println(pulsaciones);
    }
  }
}
```

Todo junto: estructura de un juego de luces

```
void loop() {
  // 1) Entradas
  if (botonPresionado()) velocidad = (velocidad + 1) % N_VEL;

  // 2) Lógica temporizada
  uint32_t ahora = millis();
  if (ahora - tPaso ⩾ PERIODOS[velocidad]) {
    tPaso = ahora;
    posicion = (posicion + 1) % N_LEDS;
  }

  // 3) Salidas
  for (uint8_t i = 0; i < N_LEDS; i++)
    digitalWrite(LEDS[i], i == posicion);
}
```

Entradas
botón con antirrebote y flanco

Lógica
estado + `millis()`

Salidas
LEDs según el estado

Resumen rápido

- **Tipos:** `int` cambia de tamaño; usar `uint8_t`, `uint32_t`…; `1UL << n`; cast tras `~`.
- **Desbordamiento:** sin signo da la vuelta; con signo es indefinido.
- **Tiempo:** `millis()` y resta: `ahora - tAnterior ⩾ PERIODO`.
- **Pulsador:** `INPUT_PULLUP`, antirrebote de 30 ms, actuar en el flanco.